



UNIVERSIDADE KIMPA VITA

Instituto Politécnico de Uíge

Licenciatura em Engenharia Informática · 3.º Ano

TRABALHO PRÁTICO DE POO II

Sistema de Gestão de Biblioteca

Desenvolvimento em C# Console Application com Modelação UML

Grupo:

1. Egas Manuel Ribeiro
2. Nzongo Oceano Nsumbu Manuel
3. Hermenegildo Wilson dos Santos Panzo

DOCENTE

Joaquim João Nsaku Ventura, MSc.

Uíge, 24 de Maio de 2026

Índice

1. Introdução	1
2. Análise de Requisitos.....	2
2.1. Requisitos Funcionais	2
2.2. Requisitos Não Funcionais	2
2.3. Casos de Utilização.....	2
3. Modelo UML — Diagrama de Classes.....	4
3.1. Classes do Sistema.....	4
3.2. Relações entre Classes	5
4. Descrição das Principais Classes	6
4.1. Classe Livro	6
4.2. Classe Leitor	6
4.3. Classe Emprestimo.....	6
4.4. Classe Catalogo.....	6
4.5. Classe GestorBiblioteca	6
5. Instruções de Compilação e Execução.....	8
5.1. Estrutura de Pastas do Projecto.....	8
5.2. Pré-requisitos	8
6.3. Passos para Compilar e Executar	8
Conclusão.....	9
Referências.....	10

1. Introdução

O presente relatório descreve o desenvolvimento do Sistema de Gestão de Biblioteca, projecto prático realizado no âmbito da Unidade Curricular de Programação Orientada a Objectos II com C#, ministrada no 3.º Ano da Licenciatura em Engenharia Informática do Instituto Politécnico da Universidade Kimpa Vita.

O sistema foi concebido para responder às necessidades de uma biblioteca municipal que necessita de controlar o seu acervo bibliográfico, gerir leitores registados e supervisionar o ciclo completo de empréstimo e devolução de obras. A solução foi implementada sob a forma de C# Console Application, sem recurso a bases de dados externas, utilizando exclusivamente colecções genéricas em memória.

A implementação aplicou os princípios nucleares da Programação Orientada a Objectos — encapsulamento, composição, polimorfismo e interfaces — e foi acompanhada de um diagrama de classes UML que modela a arquitectura do sistema antes da sua codificação.

2. Análise de Requisitos

2.1. Requisitos Funcionais

Requisitos funcionais são as especificações que definem o que um sistema, software ou produto deve fazer. Eles descrevem as ações concretas, comportamentos e funções que a aplicação deve executar para atender às necessidades do usuário e do negócio. O nosso sistema deve disponibilizar ao utilizador:

- **Registrar, editar e remover livros** com título, ISBN, autor, ano e número de exemplares.
- **Cadastrar e gerir leitores** com validação de dados pessoais.
- **Realizar e registrar empréstimos** com data prevista de devolução (14 dias por defeito).
- **Processar devoluções** com cálculo automático de multa por atraso (50 Kz/dia).
- Pesquisar livros por título, autor ou ISBN.
- Listar empréstimos em aberto e histórico completo por leitor.

2.2. Requisitos Não Funcionais

Requisitos Não Funcionais (RNFs) definem **como** um sistema deve funcionar, e não o que ele deve fazer. Eles estabelecem critérios de qualidade, desempenho, segurança e restrições que garantem que o software seja útil, seguro e eficiente.¹

- Dados geridos exclusivamente em memória com colecções genéricas (sem base de dados).
- Validação de todos os dados introduzidos pelo utilizador nas propriedades das classes.
- Código organizado em pastas e namespaces distintos (Models, Services, Interfaces, Exceptions, Enums).
- Mínimo de três excepções personalizadas derivadas de System.Exception.
- Separação clara de responsabilidades — Program.cs apenas orquestra o menu.

2.3. Casos de Utilização

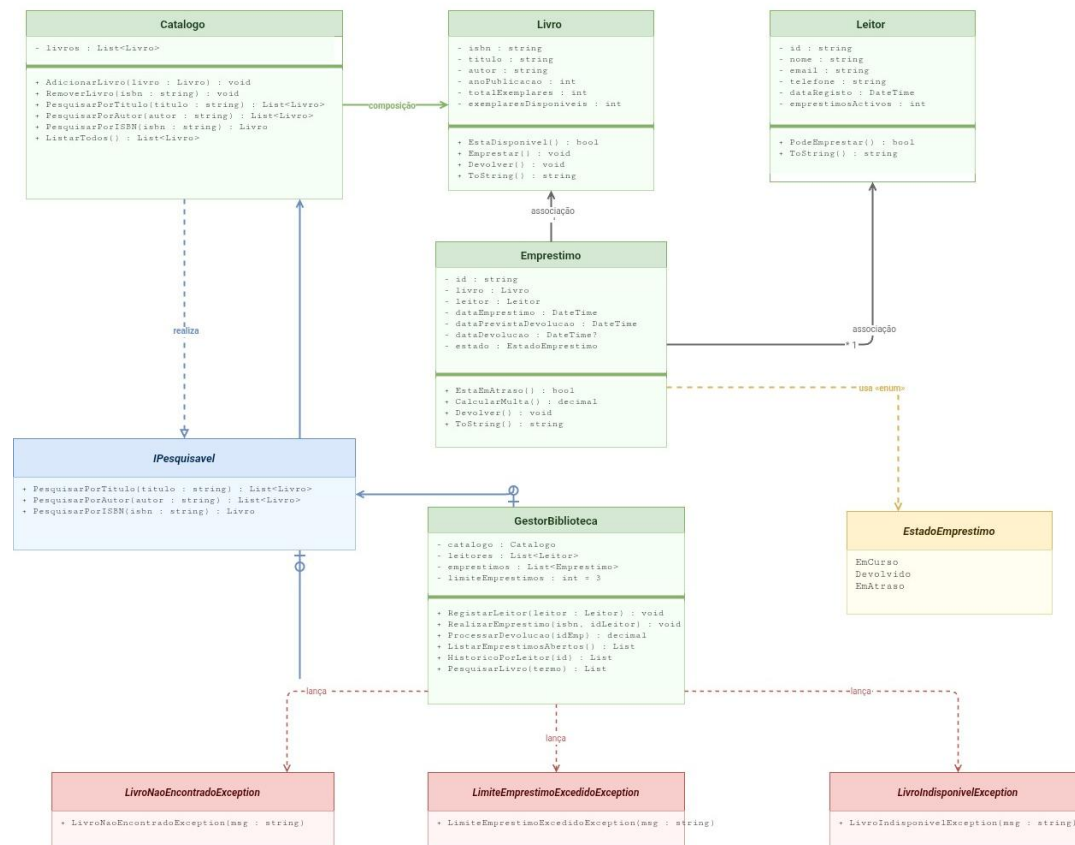
Os principais casos de utilização identificados são:

¹ <https://visuresolutions.com/pt/guia-de-escolas/requisitos-funcionais-vs-n%C3%A3o-funcionais/>

Caso de Uso	Actor	Descrição
Gerir Livros	Bibliotecário	Adicionar, editar, remover e pesquisar livros no acervo.
Gerir Leitores	Bibliotecário	Registar e consultar leitores da biblioteca.
Realizar Empréstimo	Bibliotecário	Emprestar um livro disponível a um leitor elegível.
Processar Devolução	Bibliotecário	Registar devolução e calcular multa por atraso.
Pesquisar Acervo	Bibliotecário	Pesquisar livros por título, autor ou ISBN.
Consultar Histórico	Bibliotecário	Listar empréstimos abertos e histórico por leitor.

3. Modelo UML — Diagrama de Classes

O diagrama de classes UML foi elaborado antes da implementação e reflecte fielmente a arquitectura do sistema. Apresenta-se a seguir o diagrama construído e a descrição textual das relações representadas no diagrama.



Img. 1 – Diagrama UML do sistema criado com a ferramenta draw.io²

3.1. Classes do Sistema

Classe	Tipo	Responsabilidade
Livro	Model (Entidade)	Representa um livro no acervo com stock de exemplares.
Leitor	Model (Entidade)	Representa um leitor registado com limite de empréstimos.

² https://drive.google.com/file/d/1k_PBMGFIE_AvZGoO27k48oPOxX-ww73B/view?usp=sharing

Classe	Tipo	Responsabilidade
Emprestimo	Model (Entidade)	Liga um Livro a um Leitor com datas, estado e multa.
Catalogo	Service	Gere o acervo de livros. Implementa IPesquisavel.
GestorBiblioteca	Service	Orquestra toda a lógica de negócio do sistema.
IPesquisavel	Interface	Define o contrato de pesquisa no catálogo.
EstadoEmprestimo	Enum	EmCurso, Devolvido, EmAtraso.

3.2. Relações entre Classes

Relação	Tipo	Descrição
Catalogo → Livro	Composição (1..*)	O catálogo é composto por livros. O ciclo de vida dos livros depende do catálogo.
GestorBiblioteca → Catalogo	Agregação (1)	O gestor usa o catálogo mas não controla o seu ciclo de vida.
Emprestimo → Livro	Associação (*→1)	Um empréstimo referencia um livro específico.
Emprestimo → Leitor	Associação (*→1)	Um empréstimo referencia um leitor específico.
Catalogo → IPesquisavel	Realização	O Catálogo implementa o contrato da interface IPesquisavel.
Emprestimo → EstadoEmprestimo	Dependência	O Empréstimo usa o enum para representar o seu estado.

4. Descrição das Principais Classes

4.1. Classe Livro

A classe Livro representa uma obra do acervo bibliográfico. Todos os atributos são privados e o acesso externo é feito exclusivamente através de propriedades com validação encapsulada. Os métodos Emprestar() e Devolver() controlam o stock de exemplares disponíveis, lançando exceções quando as operações não são possíveis.

Atributos principais: isbn (string), titulo (string), autor (string), anoPublicacao (int), totalExemplares (int), exemplaresDisponiveis (int).

Métodos de negócio: EstaDisponivel() : bool — verifica se há exemplares; Emprestar() : void — decrementa o stock; Devolver() : void — incrementa o stock.

4.2. Classe Leitor

A classe Leitor representa um utilizador registado na biblioteca. Define a constante LimiteMaximoEmprestimos = 3, que centraliza a regra de negócio do número máximo de empréstimos activos por leitor. O método PodeEmprestar() encapsula esta verificação, enquanto IncrementarEmprestimos() e DecrementarEmprestimos() mantêm o contador actualizado.

4.3. Classe Emprestimo

A classe Emprestimo liga um Livro a um Leitor com controlo temporal. O atributo DataDevolucao é do tipo DateTime? (nullable), sendo null enquanto o livro não for devolvido. O ID é gerado automaticamente com Guid.NewGuid(). O método CalcularMulta() aplica 50 Kz por cada dia de atraso. O estado do empréstimo é representado pelo enum EstadoEmprestimo.

4.4. Classe Catalogo

O Catálogo gere o acervo de livros através de um Dictionary<string, Livro>, onde a chave é o ISBN. Esta estrutura permite acesso directo em tempo O(1). Implementa a interface IPesquisavel, fornecendo pesquisa por título, autor e ISBN com comparação insensível a maiúsculas/minúsculas (StringComparison.OrdinalIgnoreCase).

4.5. Classe GestorBiblioteca

O GestorBiblioteca é a classe central que orchestra toda a lógica de negócio. Delega as operações sobre livros no Catálogo e gere directamente leitores e empréstimos. O método `RealizarEmprestimo()` verifica em sequência: existência do livro, disponibilidade de exemplares, existência do leitor e limite de empréstimos activos — lançando a excepção adequada em cada caso.

5. Instruções de Compilação e Execução

5.1. Estrutura de Pastas do Projecto

```
# File Tree: Gestao_de_Biblioteca
**Generated:** 24/05/2026, 14:28:01
**Root Path:**
`/home/hermenegildo/Documents/ProjectosC#/Gestao_de_Biblioteca`
├── Enums
│   └── EstadoEmprestimo.cs
├── Exceptions
│   ├── LimiteEmprestimoExcedidoException.cs
│   ├── LivroIndisponivelException.cs
│   └── LivroNaoEncontradoException.cs
├── Interfaces
│   └── IPesquisavel.cs
├── Models
│   ├── Emprestimo.cs
│   ├── Leitor.cs
│   └── Livro.cs
├── Services
│   ├── Catalogo.cs
│   └── GestorBiblioteca.cs
├── docs
│   ├── Diagrama Sistema de Gestão de Biblioteca.jpg
│   ├── Proposta_Avaliacao_P002_II.pdf
│   └── Relatorio_SistemaBiblioteca.pdf
├── .gitignore
├── Gestao_de_Biblioteca.csproj
├── LICENSE
├── Program.cs
└── README.md
---
```

Generated by FileTree Pro Extension

5.2. Pré-requisitos

- .NET SDK 6.0 ou superior (<https://dotnet.microsoft.com>)
- Visual Studio 2022 / Visual Studio Code com extensão C# Dev Kit
- Sistema operativo: Windows 10/11, macOS ou Linux

6.3. Passos para Compilar e Executar

1. Abrir o terminal na pasta raiz do projecto (GestãoBiblioteca/).
2. Compilar o projecto com o comando: `dotnet build`
3. Executar o projecto com o comando: `dotnet run`
4. O menu principal aparece na consola e permite navegar pelas funcionalidades.

Conclusão

O desenvolvimento do Sistema de Gestão de Biblioteca permitiu ao grupo consolidar e aplicar, num contexto prático e próximo da realidade profissional, os conceitos fundamentais da Programação Orientada a Objectos em C#.

A modelação prévia com o diagrama de classes UML revelou-se essencial: ao definir antecipadamente as entidades, atributos, métodos e relações, a fase de implementação tornou-se mais organizada e consistente, evitando retrabalho e inconsistências no código.

A aplicação dos princípios de encapsulamento — com atributos privados e validação encapsulada nas propriedades — garantiu a integridade dos dados em todas as classes. A utilização do `Dictionary<string, Livro>` no Catálogo demonstrou a importância da escolha da estrutura de dados adequada para o contexto: acesso directo por ISBN em tempo constante $O(1)$.

O uso de excepções personalizadas (`LivroNaoEncontradoException`, `LivroIndisponivelException` e `LimiteEmprestimoExcedidoException`) tornou o sistema robusto e expressivo, permitindo identificar com precisão a natureza de cada falha de negócio e tratá-la de forma controlada com blocos `try-catch-finally`.

A interface `IPesquisavel` demonstrou o valor do polimorfismo por realização: ao referenciar o Catálogo através do tipo da interface, o código cliente fica desacoplado da implementação concreta, tornando o sistema extensível sem necessidade de modificar o código existente.

Como dificuldades encontradas, destaca-se a gestão correcta do tipo `DateTime?` (nullable) para representar a ausência de devolução, e a sincronização dos contadores entre as classes `Livro`, `Leitor` e `Emprestimo` durante as operações de empréstimo e devolução. Ambas foram resolvidas com encapsulamento rigoroso e métodos de negócio específicos em cada classe.

Referências

- Ventura, J. J. N. (2025/2026). Proposta Académica de Avaliação Prática — Desenvolvimento de Projectos em C# Console Application. Instituto Politécnico da Universidade Kimpa Vita.
- Ventura, J. J. N. (2025/2026). Guia de Laboratório — Programação Orientada a Objectos II: Interfaces, Colecções e Tratamentos de Excepções. Instituto Politécnico da Universidade Kimpa Vita.
- Microsoft Corporation. (2024). C# documentation. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/csharp/>
- Microsoft Corporation. (2024). System.Collections.Generic Namespace. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Visure Solutions. (s.d.). *Requisitos funcionais VS não funcionais (com exemplos)*. Guia de Esmolas. visuresolutions.com